



**Mondragon
Unibertsitatea**

**Escuela Politécnica
Superior**

Reinforcement Learning -RL-

Deep Reinforcement Learning -Policy Gradient methods-

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu

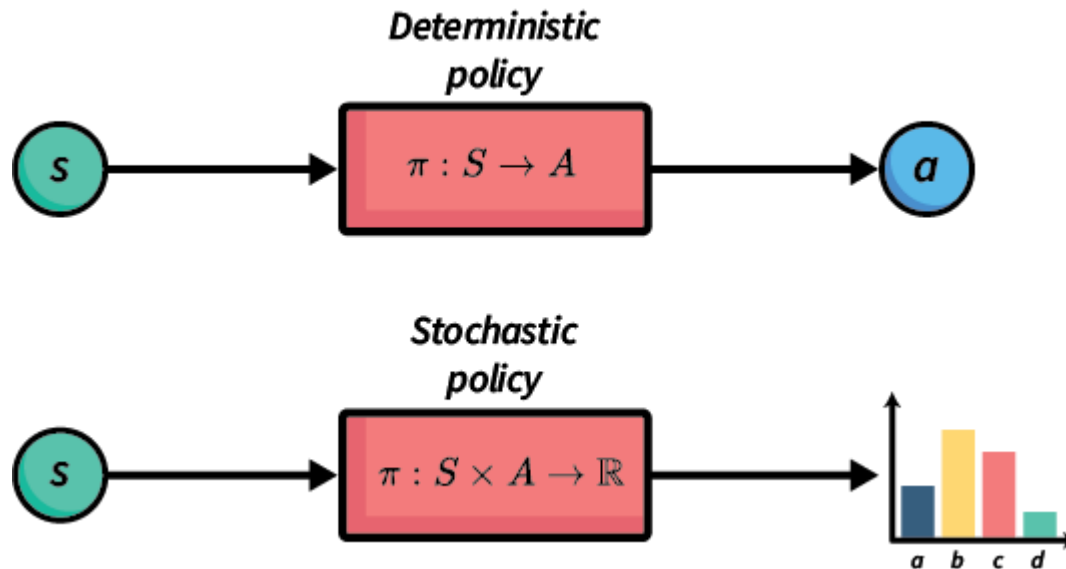
REINFORCE algorithm

This section is based on:

Mutual Information. (2023, May). *Policy Gradient Methods | Reinforcement Learning Part 6* [Video]. YouTube. https://www.youtube.com/watch?v=e20EY4tFC_Q

REINFORCE algorithm

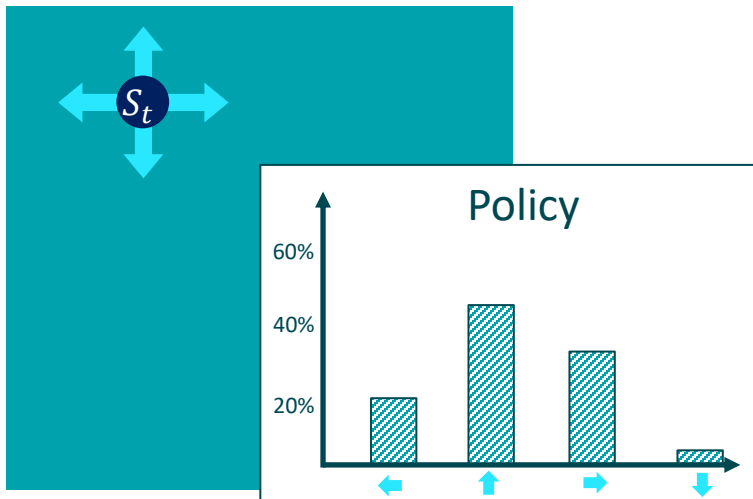
- **Policy gradient methods**
 - Actually, we are learning Q-values to stimate the best policy.
 - Can we learn the policy directly? YES!!
- **Stochastic policies**
 - A **stochastic policy** means that for a given state, the agent **chooses actions probabilistically** rather than deterministically.



REINFORCE algorithm

- **Policy gradient methods**

- So far we have seen action-value methods.
- A value function may still be used to learn the policy parameter, but is not required for action selection.
- Instead of using state-value actions, we will use probabilities for action selection.
- Let's assume the following example of a continuous state space



$$\pi(a|s, \theta) = \Pr\{A_t = a \mid S_t = s, \theta_t = \theta\}$$

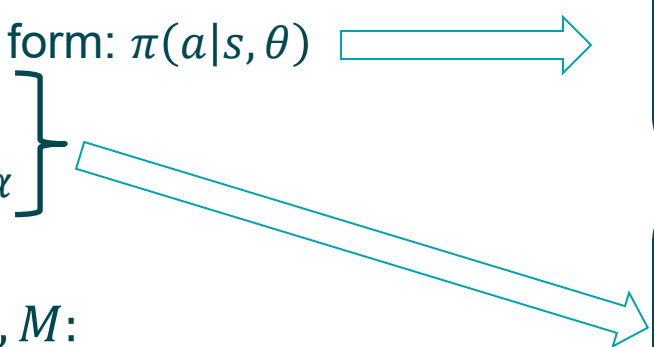
In this case neural network weights

We need to estimate a θ to determine directly the policy

REINFORCE algorithm

- **REINFORCE algorithm**

- Its a Monte Carlo algorithm

- Functional form: $\pi(a|s, \theta)$
 - Initial θ
 - Step size α
- 

We use something to estimate θ .

- Simple linear model
- Neural network

- For $m = 1, \dots, M$:

- Sample:
 - » $s_0^m, a_0^m, r_1^m \dots a_{T_m-1}^m, r_{T_m}^m$
 - » For $t = 0, \dots, T_m - 1$:
 - $\theta \leftarrow \theta + \alpha(????)$

Upgrades will be applied to θ to take us from an arbitrary position to some place that gives us high rewards.

REINFORCE algorithm

- REINFORCE algorithm**

1. Imagine the following image, where we have two fixed points (states) P_{t_A} and P_{t_B} , each with each own fixed action probabilities.
2. We can use these to form action probabilities for any point S_t in the state space looking to the distances between the S_t and P_t points.
3. So, if we are more close to P_{t_A} the action probabilities of S_t will be more similar to P_{t_A} .

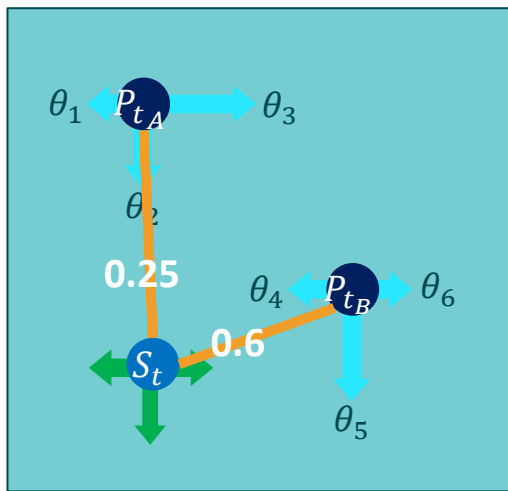


Fig1

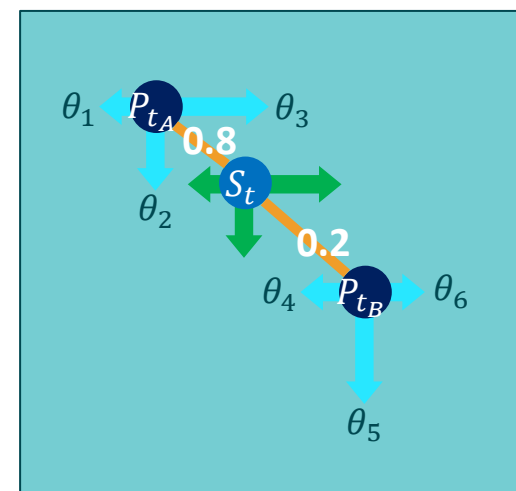
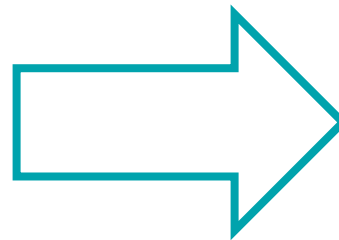
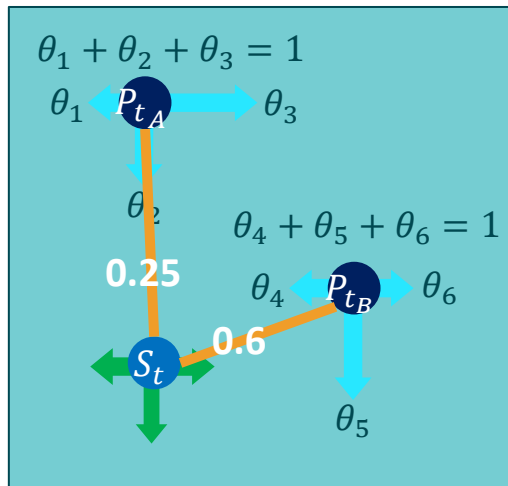


Fig2

REINFORCE algorithm

• REINFORCE algorithm

- But we have a complication, our θ are action probabilities. So, they should sum 1. So, as we move through the state space, it becomes complicated to manage this restriction.



We convert these probabilities to some scalar values

These values are unconstrained

$$f\left(\begin{bmatrix} \theta_l \\ \theta_r \end{bmatrix}\right) = \begin{bmatrix} p_l \\ p_d \\ p_r \end{bmatrix}$$

Where:

$$p_l + p_d + p_r = 1$$

$$p_l \geq 0; \quad p_d \geq 0; \quad p_r \geq 0$$



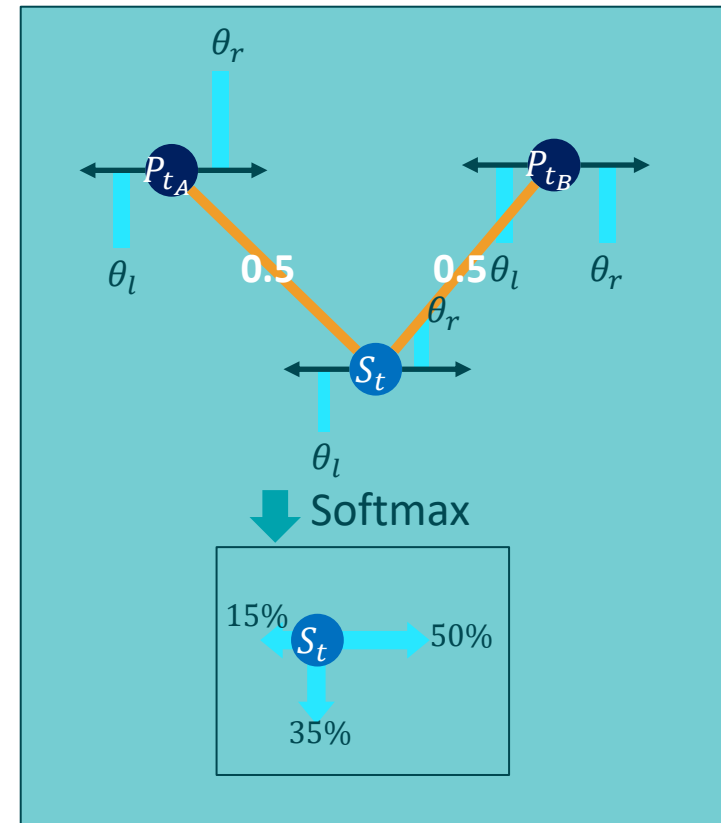
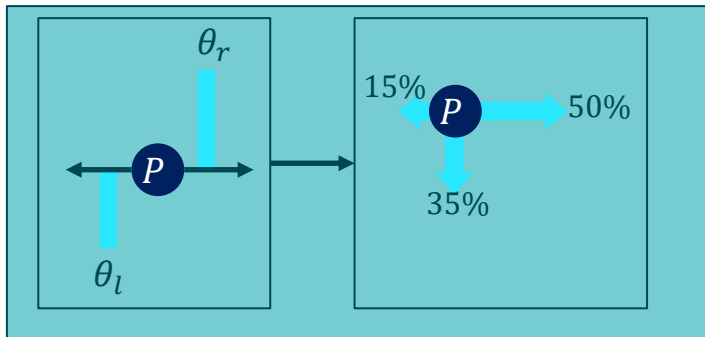
Softmax:

$$f\left(\begin{bmatrix} \theta_l \\ \theta_r \end{bmatrix}\right) = \begin{bmatrix} \frac{e^{\theta_l}}{c}, \frac{1}{c}, \frac{e^{\theta_r}}{c} \end{bmatrix}^T$$

$$c = e^{\theta_l} + 1 + e^{\theta_r}$$

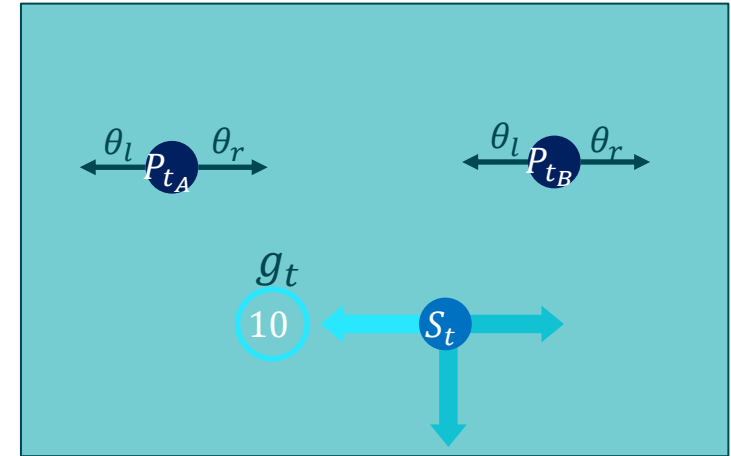
REINFORCE algorithm

- **REINFORCE algorithm**
 - We can see the slide before from this perspective



REINFORCE algorithm

- **Update**
 - Suppose we obtain a return of 10 at the end of an episode.
 - The rule says, increase the probability of an action a in a state s in proportion to the return. If it is negative decrease the probability.



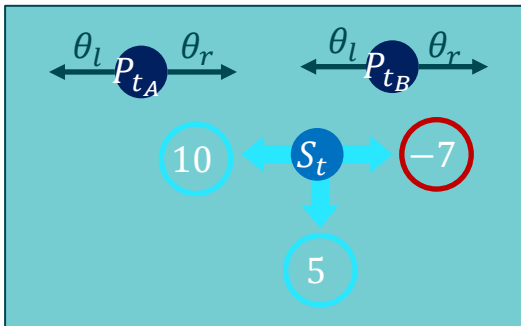
$$\theta \leftarrow \theta + \alpha g_t^m \nabla \pi(a_t^m, s_t^m, \theta)$$

The gradient is the direction to push θ so that the probability of action a in state s increases to the maximum.

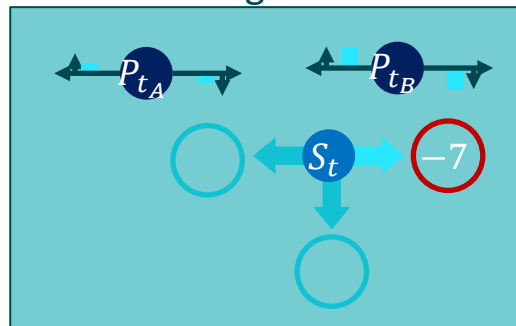
REINFORCE algorithm

- Let's assume that the agent is in the state S_t repeatedly

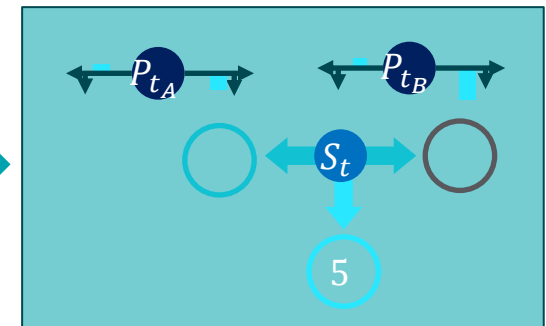
Initialize $\theta = 0$



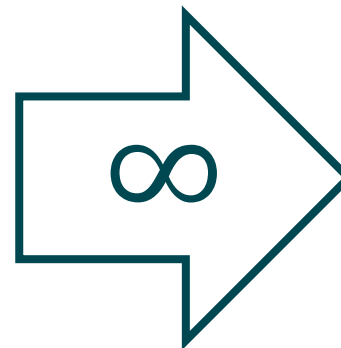
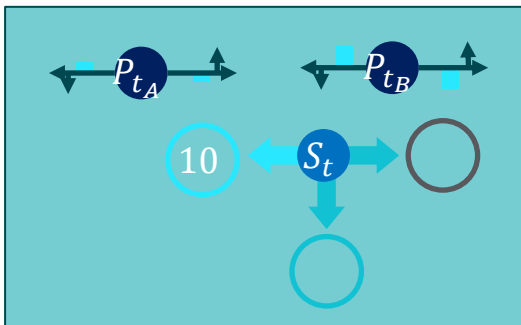
Take action right



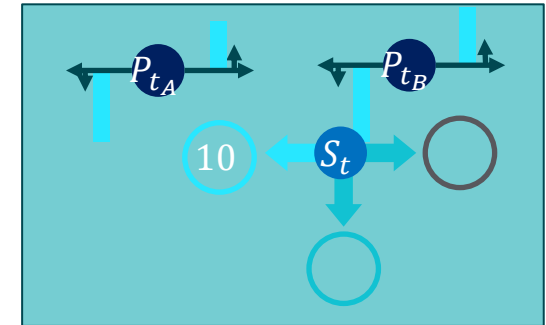
Take action down



Take action left



Take action left

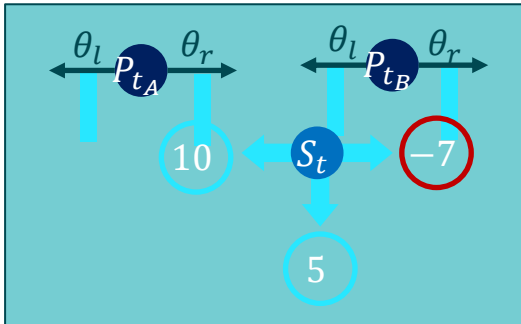


$$\theta \leftarrow \theta + \alpha g_t^m \nabla \pi(a_t^m, s_t^m, \theta)$$

REINFORCE algorithm

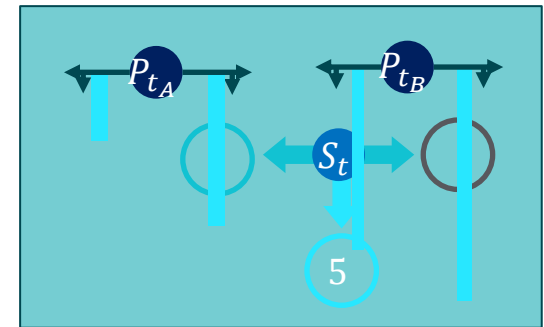
- Something is wrong... Imagine we start with the following initialization

Initialize $\theta \neq 0$



- This causes the downward action to begin to dominate.
- And even if it is not the best action, as it is most often selected, taking the action on the left does not compensate for the upgrades.

Take action down



$$\theta \leftarrow \theta + \alpha g_t^m \frac{\nabla \pi(a_t^m, s_t^m, \theta)}{\pi(a_t^m | s_t^m, \theta)}$$

Divide by the probability of the action

$$\theta \leftarrow \theta + \alpha \gamma^t g_t^m \nabla \ln(a_t^m | s_t^m, \theta)$$

REINFORCE algorithm

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

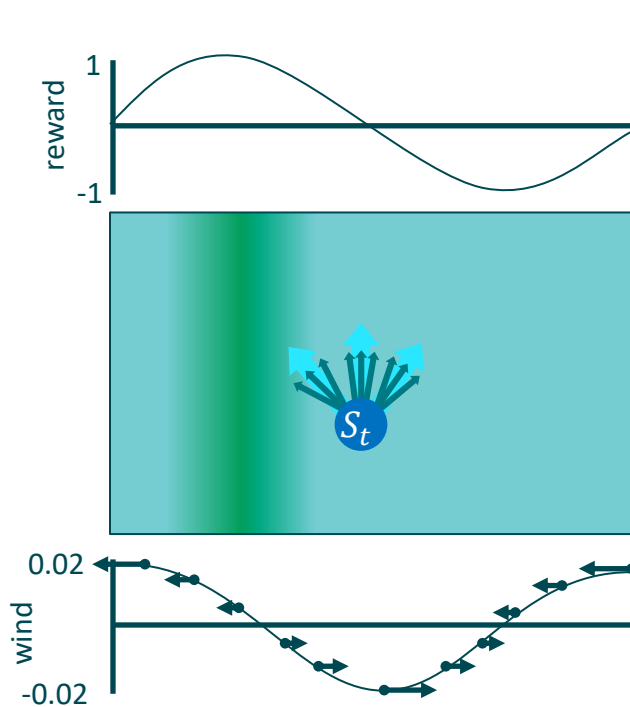
$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta)$$

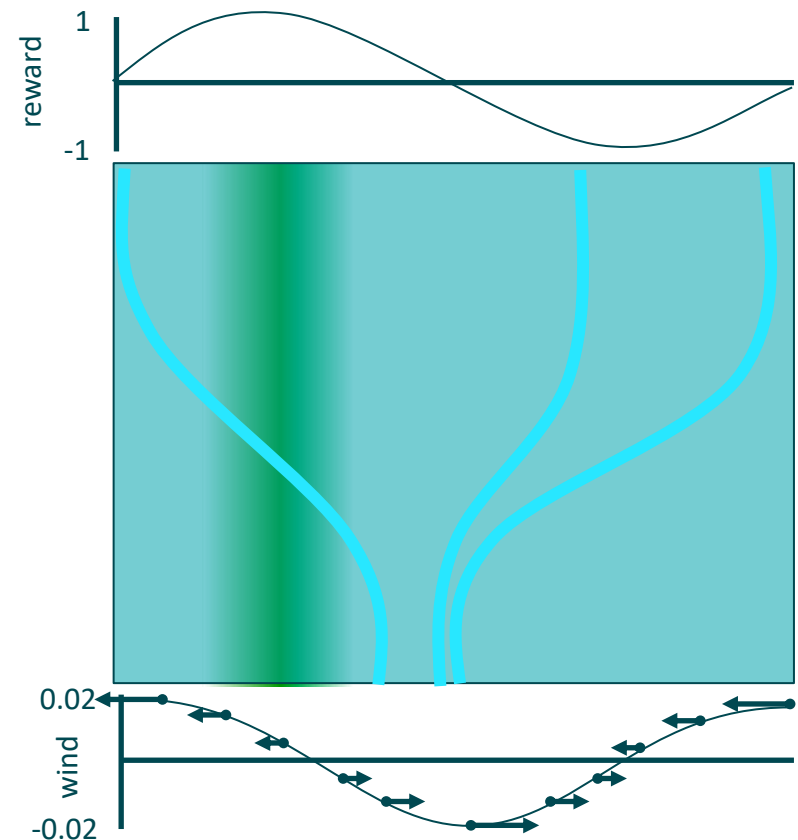
Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT Press, Cambridge, MA, 2018.

REINFORCE algorithm

- In a continuous state space we can have a wind



Sometimes we have wind: it refers to a dynamic factor that influences the state transitions, this affects the system behavior over time



REINFORCE algorithm

- Now suppose that we add 4 to all rewards. So all rewards are positive. **This slows down learning**

$$\theta \leftarrow \theta + \alpha \gamma^t g_t^m \nabla \ln(a_t^m | s_t^m, \theta)$$

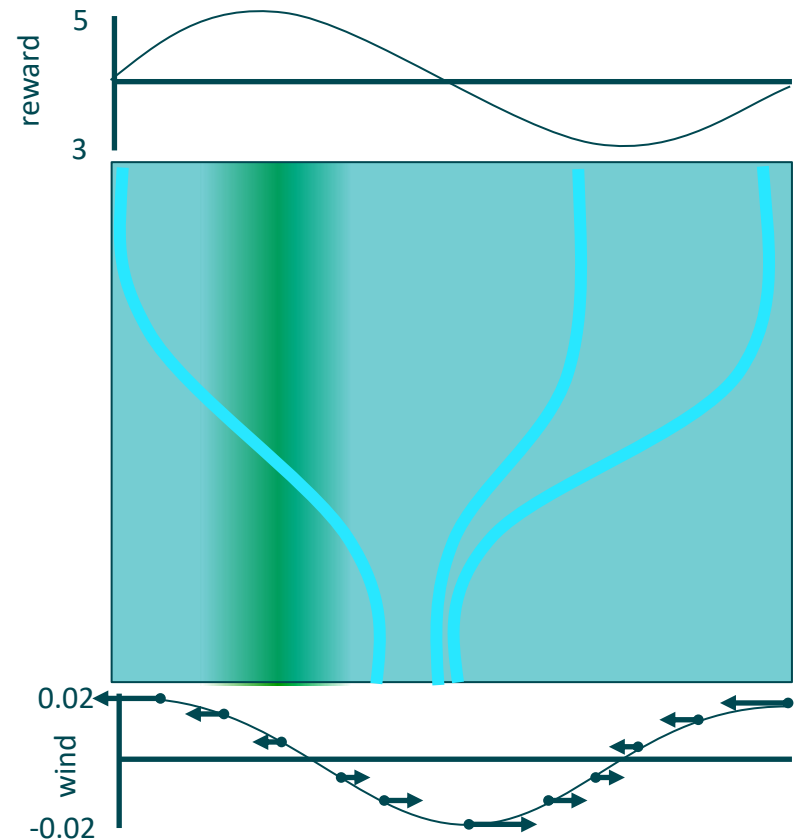
This is the direction where we want to push our agent learning

So all $a_t^m | s_t^m$ will be more likely

The agent never learns to NO do things



We replace g_t^m by $g_t^m - b(s_t^m)$



REINFORCE algorithm

- The baseline $b(s_t^m)$:

- $b(s_t^m) = \hat{v}(s_t^m, w)$ —————→

Estimate of the value

- Then:

- $g_t^m - \hat{v}(s_t^m, w)$

- » Positive for $g_t^m > \hat{v}(s_t^m, w)$ actions.
 - » Negative for $g_t^m < \hat{v}(s_t^m, w)$ actions.

$$\theta \leftarrow \theta + \alpha \gamma^t (g_t^m - b(s_t^m)) \nabla \ln(a_t^m | s_t^m, \theta)$$

And this variance reduction makes REINFORCE become Actor Critic

REINFORCE algorithm

REINFORCE with Baseline (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Algorithm parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

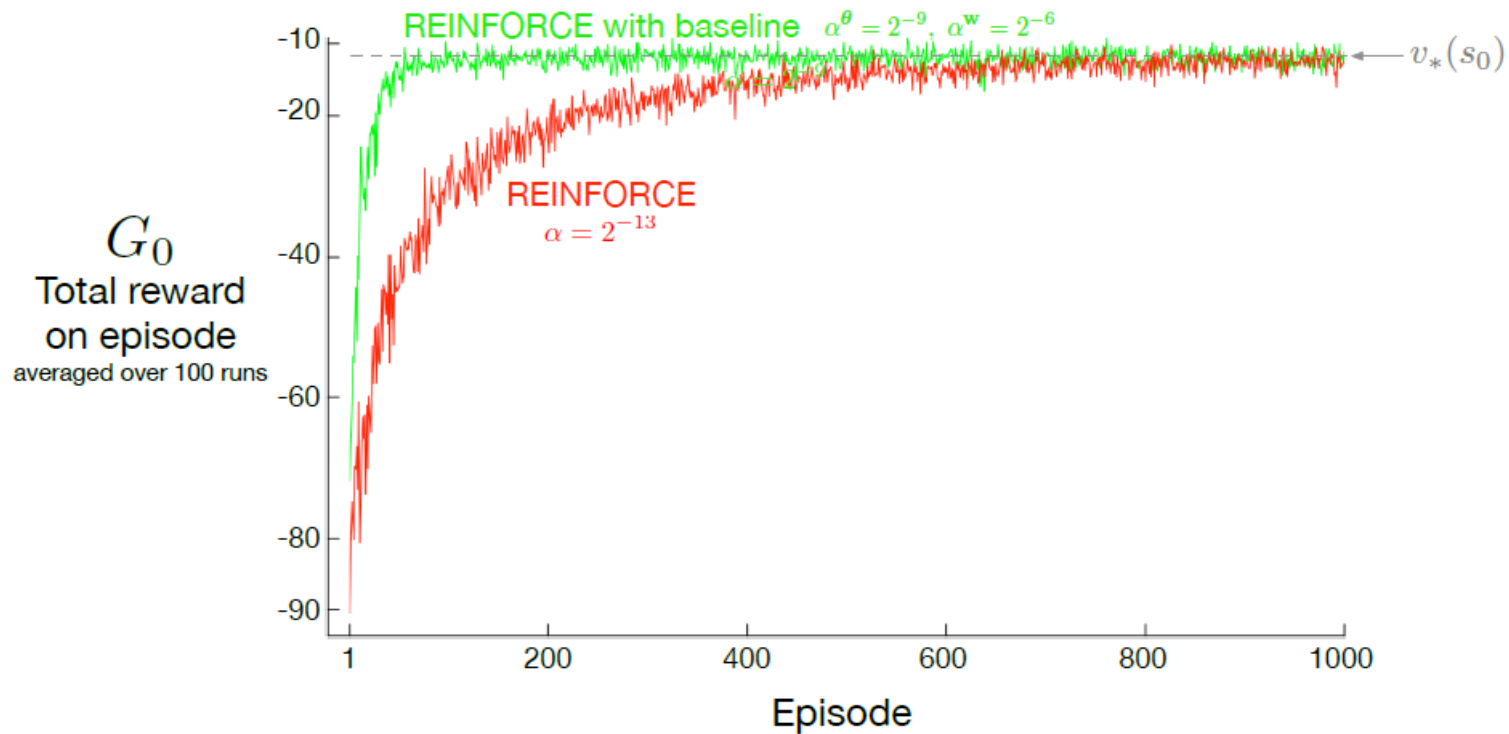
$$\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$$

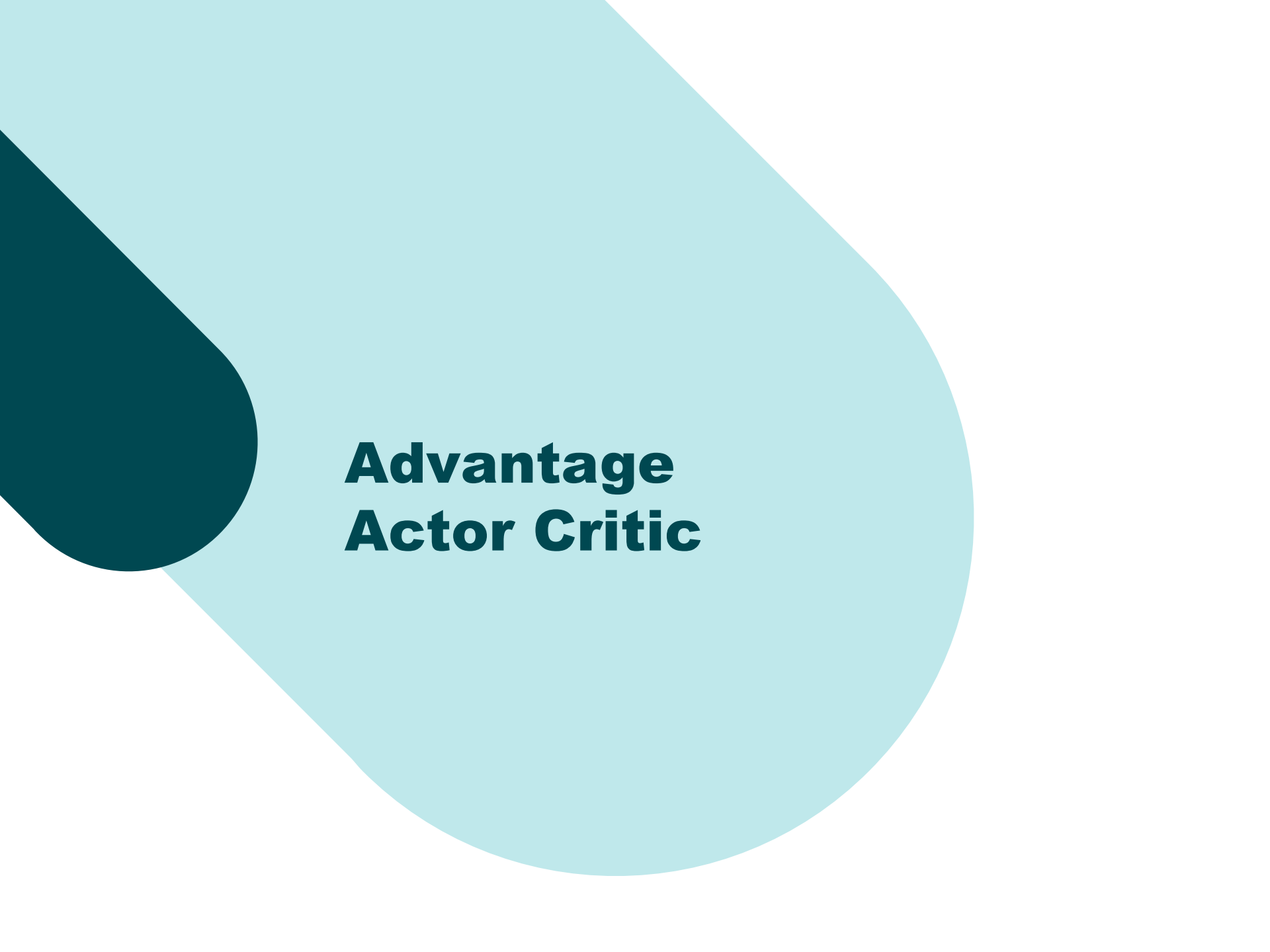
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w})$$

$$\theta \leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t|S_t, \theta)$$

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT Press, Cambridge, MA, 2018.

REINFORCE algorithm



The background features a large, light blue circle on the left side, partially overlapping a dark teal shape that extends from the left edge. The rest of the background is white.

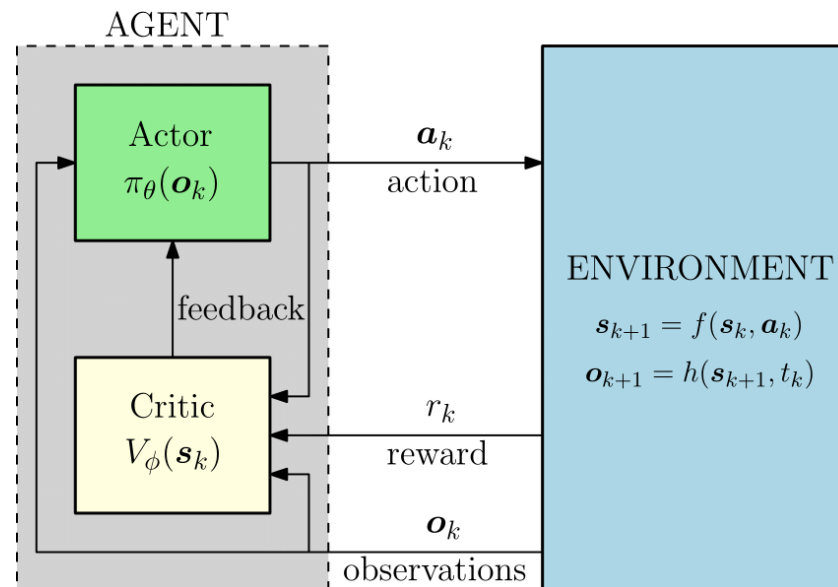
Advantage Actor Critic

Advantage Actor Critic

- In REINFORCE as it is a Monte Carlo method we need all the episode to estimate the return.
 - This adds variance in policy gradient estimator
 - Because the environment (what happens during an episode) and policy (what we do) are stochastic, the same starting point can lead to very different results.
 - This can make the returns very different, and the same starting point can lead to very different returns.
 - Because of this, the return starting at the same state can vary a lot across different instances.

Advantage Actor Critic

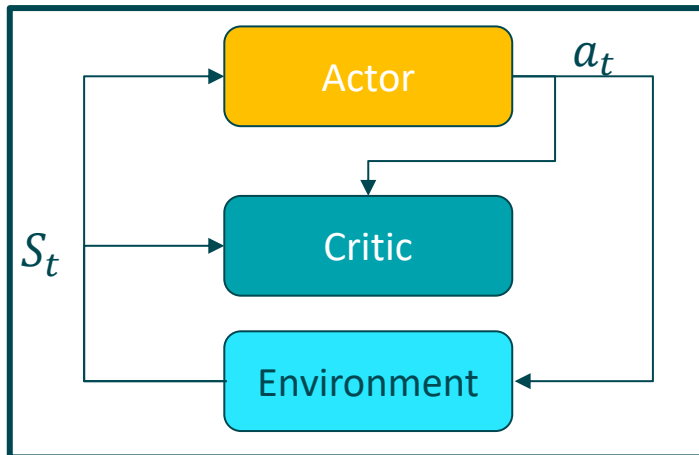
- **Actor:** Learns a policy that maps states to actions, selecting actions based on the current state of the environment.
- **Critic:** Learns a value function that estimates the expected return (total reward) from a given state under the current policy.



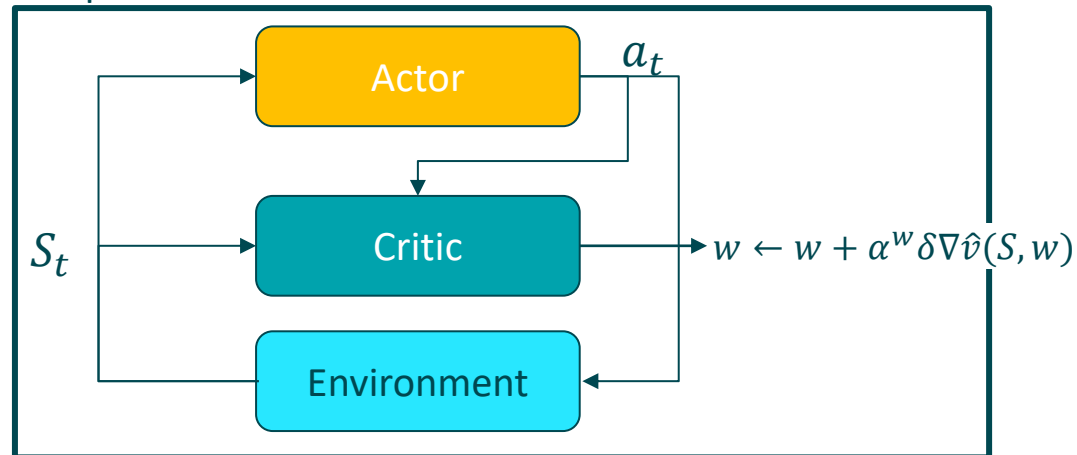
Advantage Actor Critic

- Step 1:
 - At each timestep, t , we get the current state S_t from the environment and pass it as input through our Actor and Critic.
 - Our Policy takes the state and outputs an action .
- Step 2:
 - The Critic takes that action also as input and, using S_t and A_t , **computes the value of taking that action at that state: the Q-value.**

Step 1



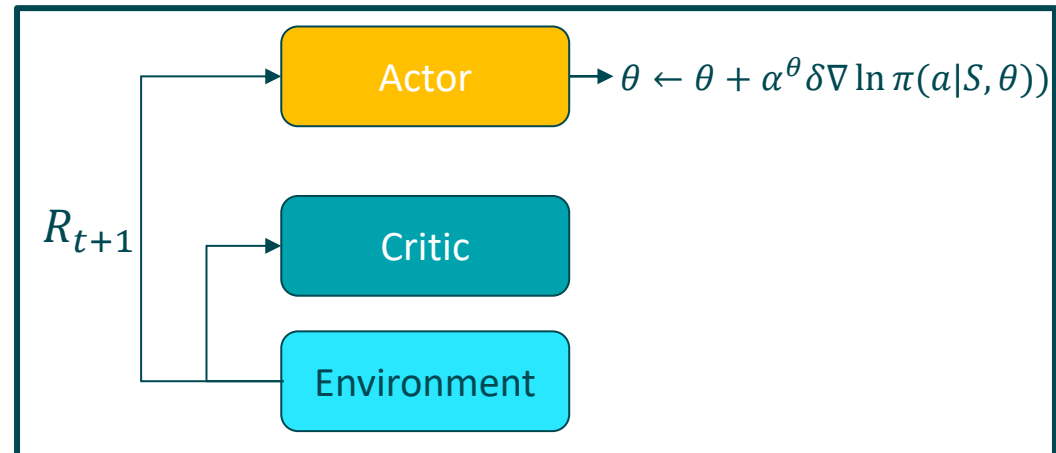
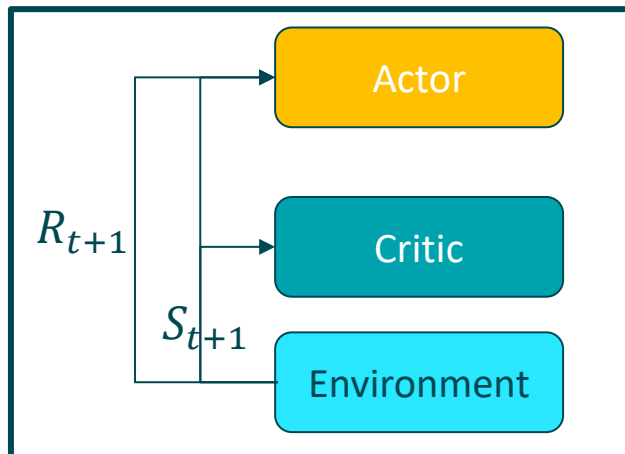
Step 2



Advantage Actor Critic

- Step 3:
 - The action a_t performed in the environment outputs a new state S_{t+1} and a reward.
- Step 4:
 - The Actor updates its policy parameters

Step 3



Advantage Actor Critic

One-step Actor–Critic (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Initialize S (first state of episode)

$I \leftarrow 1$

 Loop while S is not terminal (for each time step):

$A \sim \pi(\cdot|S, \theta)$

 Take action A , observe S', R

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, then $\hat{v}(S', \mathbf{w}) \doteq 0$)

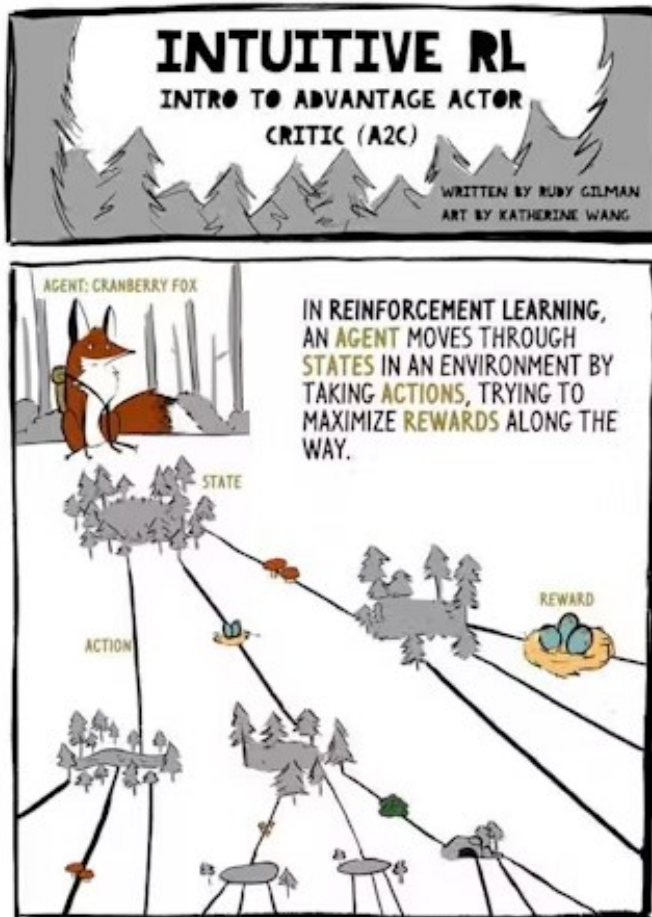
$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S, \mathbf{w})$

$\theta \leftarrow \theta + \alpha^{\theta} I \delta \nabla \ln \pi(A|S, \theta)$

$I \leftarrow \gamma I$

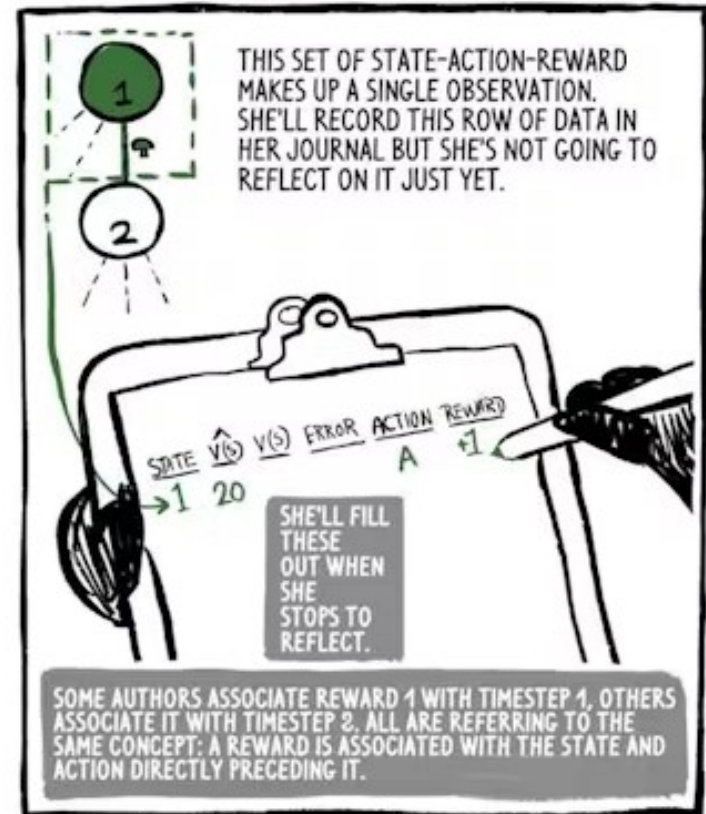
$S \leftarrow S'$

Advanced Actor Critic



<https://hackernoon.com/intuitive-rl-intro-to-advantage-actor-critic-a2c-4ff545978752>

Advanced Actor Critic

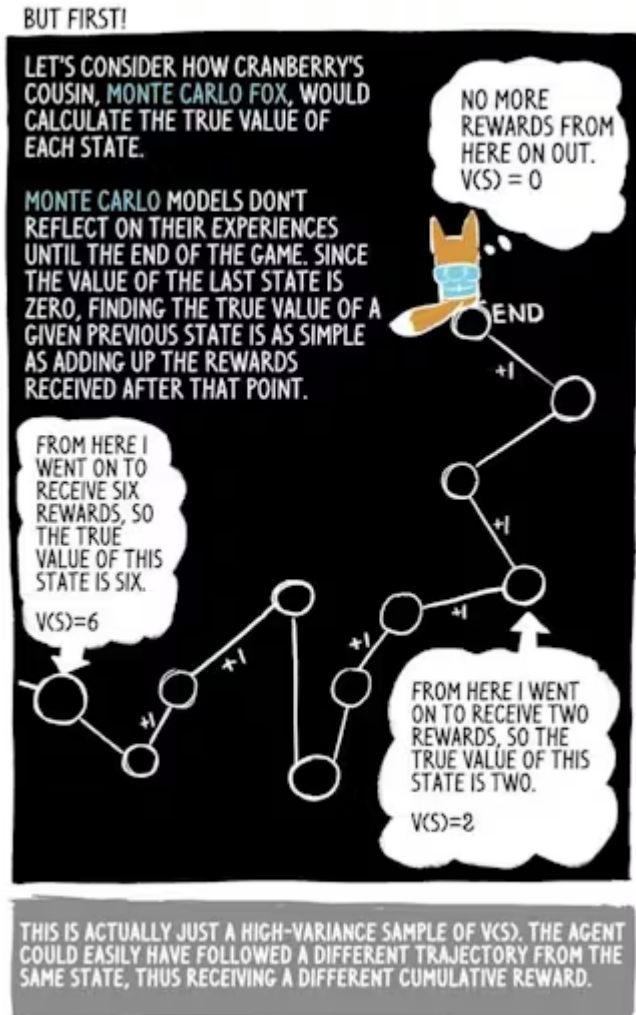
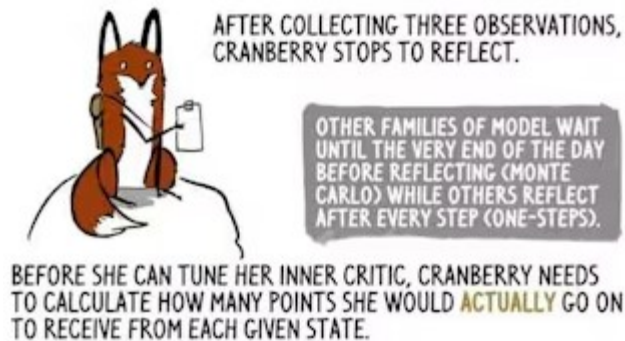


Advanced Actor Critic



<https://hackernoon.com/intuitive-rl-intro-to-advantage-actor-critic-a2c-4ff545978752>

Advanced Actor Critic



<https://hackernoon.com/intuitive-rl-intro-to-advantage-actor-critic-a2c-4ff545978752>

Advanced Actor Critic

BUT CRANBERRY IS GOING TO STOP AND REFLECT MANY TIMES BEFORE THE DAY IS OVER. SHE WON'T KNOW HOW MANY POINTS SHE'LL ACTUALLY GO ON TO RECEIVE FROM EACH STATE UNTIL THE END OF THE GAME BECAUSE THE END OF THE GAME IS HOURS AWAY!

THIS IS WHERE SHE DOES SOMETHING REALLY CLEVER—CRANBERRY FOX ESTIMATES HOW MANY POINTS SHE'LL GO ON TO GET FROM THE LAST STATE IN THE SET. LUCKILY, SHE HAS A STATE-ESTIMATOR BUILT RIGHT IN—HER CRITIC!

THIS CLEARING LOOKS DECENT, I'D SAY I'LL GATHER 18 MORE FOX POINTS BEFORE THE DAY IS DONE.

WITH THIS ESTIMATE IN HAND, CRANBERRY CAN CALCULATE THE "TRUE" VALUES OF THE PRECEDING STATES EXACTLY AS MONTE CARLO FOX DID.

MONTE CARLO FOX ESTIMATED THE TARGET "LABELS" BY DOING A ROLLOUT OF A TRAJECTORY AND ADDING UP THE REWARDS FROM EACH STATE ONWARDS. A2CS TRUNCATE THIS TRAJECTORY AND REPLACE IT WITH AN ESTIMATE FROM THEIR CRITIC. THIS BOOTSTRAPPING BRINGS DOWN THE VARIANCE OF THE ESTIMATE AND ALLOWS A2CS TO OPERATE CONTINUOUSLY, THOUGH AT THE COST OF INTRODUCING A BIT OF BIAS.

T=1
 T=2
 T=3
 T=4

REWARDS ARE OFTEN DISCOUNTED TO REFLECT THE FACT THAT A REWARD NOW IS BETTER THAN A REWARD IN THE FUTURE. TO KEEP THINGS SIMPLE, CRANBERRY ISN'T DISCOUNTING HER REWARDS.

NOW CRANBERRY CAN GO THROUGH EACH ROW OF DATA AND COMPARE HER STATE-VALUE PREDICTIONS TO THEIR ACTUAL VALUES. SHE USES THE DIFFERENCE BETWEEN THESE NUMBERS TO HONE HER PREDICTION SKILLS. EVEN THREE STEPS INTO THE DAY, CRANBERRY HAS GATHERED VALUABLE EXPERIENCES WORTH REFLECTING ON.

STATE	V(0)	V(1)	ERROR	ACTION	REWARDS
1	20	41	21	A	+1
2	19	40	21	C	+20
3	22	20	-2	B	+2
4	18				

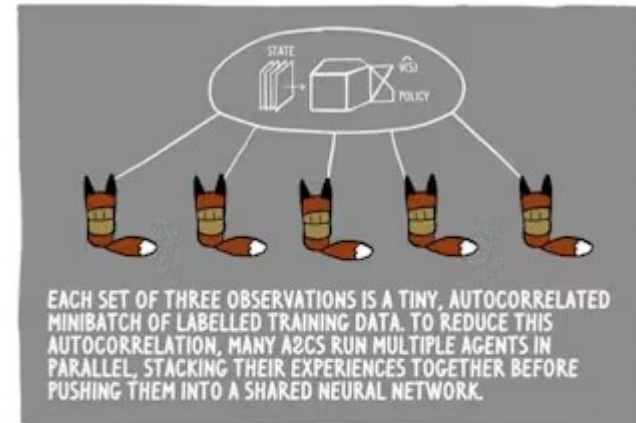
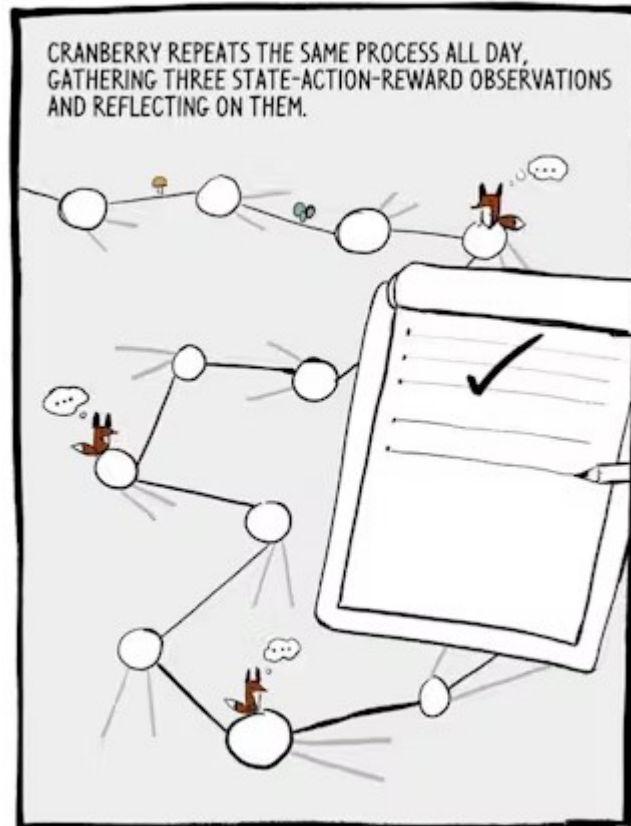
I WAS WAY OFF FOR STATE 1 AND 2! WHAT DID I MISS?

AHA! NEXT TIME I SEE FEATHERS LIKE THAT, I KNOW TO INCREASE V(S).

IT MAY SEEM CRAZY THAT CRANBERRY IS ABLE TO USE HER ESTIMATE OF V(S) AS THE GROUND TRUTH TO COMPARE HER OTHER PREDICTIONS AGAINST. BUT ANIMALS (US INCLUDED) DO THIS ALL THE TIME! IF YOU FEEL LIKE THINGS ARE GOING WELL, NO NEED TO WAIT TO REINFORCE THE ACTIONS THAT LED YOU THERE.

BY TRUNCATING OUR CALCULATED RETURNS AND REPLACING THEM WITH A BOOTSTRAPPED ESTIMATE, WE'VE TRADED SOME OF THE VARIANCE OF THE MONTE CARLO MODEL FOR A BIT OF BIAS. RL MODELS GENERALLY SUFFER FROM HIGH-VARIANCE (IMAGINE ALL THE POSSIBLE TRAJECTORIES), SO THIS IS USUALLY A WORTHWHILE EXCHANGE.

Advanced Actor Critic



THE DAY IS ALMOST OVER. ONLY TWO STEPS TO GO.

AS WE SAW EARLIER, CRANBERRY'S ACTION RECOMMENDATIONS ARE EXPRESSED AS PERCENTAGE CONFIDENCES ABOUT HER OPTIONS. INSTEAD OF SIMPLY TAKING THE HIGHEST-CONFIDENCE CHOICE, CRANBERRY SAMPLES FROM THIS ACTION DISTRIBUTION. THIS ENSURES SHE DOESN'T ALWAYS SETTLE FOR SAFE BUT POTENTIALLY MEDIOCRE ACTIONS.

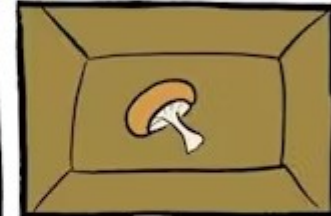
Advanced Actor Critic



Advanced Actor Critic



SOMETIMES AN AGENT WILL FIND ITSELF IN A STATE WHERE ALL ACTIONS LEAD TO NEGATIVE OUTCOMES. A2CS, HOWEVER, ARE EXCELLENT AT MAKING THE MOST OF BAD SITUATIONS.



AS THE SUN SETS, CRANBERRY REFLECTS ON THIS LAST SET OF DECISIONS.

WE TALKED ABOUT HOW CRANBERRY TUNES HER INNER CRITIC. BUT HOW DOES SHE ADJUST HER INNER ACTOR? HOW DOES SHE LEARN TO MAKE SUCH REFINED CHOICES?

Advanced Actor Critic



Advanced Actor Critic

LIKE ALL ANIMALS, AS CRANBERRY MATURES SHE'LL HONE HER ABILITY TO PREDICT STATE VALUES, GAIN INCREASING CONFIDENCE IN HER ACTION CHOICES AND FIND HERSELF SURPRISED LESS OFTEN BY REWARDS.

NOT ONLY DO RL AGENTS LIKE CRANBERRY GENERATE ALL THE DATA THEY NEED SIMPLY BY INTERACTING WITH THE ENVIRONMENT, THEY ALSO ESTIMATE THE TARGET LABELS THEMSELVES. THAT'S RIGHT, RL MODELS UPDATE PREVIOUS ESTIMATES TO BETTER MATCH NEW AND IMPROVED ESTIMATES.

AS DR. DAVID SILVER, HEAD OF THE RL GROUP AT GOOGLE DEEPMIND, PUTS IT: $AI = DL + RL$. WHEN AN AGENT LIKE CRANBERRY CAN BOOTSTRAP HER OWN INTELLIGENCE, THE POSSIBILITIES ARE LIMITLESS...

THE END



[https://hackernoon.com/intuitive-rl-](https://hackernoon.com/intuitive-rl-42800000000000000000000000000000)

Limitations

- High Variance in Policy Gradients
 - The policy gradient estimates can have **high variance**, leading to **unstable updates**.
- Inefficiency Due to Synchronous Updates
 - A2C is a **synchronous** algorithm, meaning multiple environments run in parallel, but they must **wait** for all environments to finish before updating the network.
- Sample Inefficiency
 - A2C, like other **on-policy** methods, discards old experiences after each update.



**Mondragon
Unibertsitatea**

Escuela Politécnica
Superior

Eskerrik asko
Muchas gracias
Thank you

© 2026 Mondragon Unibertsitatea.

All rights reserved. This document and its contents are protected by intellectual property rights. Its total or partial reproduction, distribution, public communication, transformation, adaptation or creation of derivative works is expressly prohibited without the written consent of the owner.

Aitor Duo Zubiaurre

aduo@mondragon.edu

Olagorta Kalea, 26
48014 Bilbo (Bizkaia)

T. +34 670 254 555

www.mondragon.edu